# CODESIGN

## LAB 2 : CUDA Programming "SUM REDUCTION"

Report Deadline: **May 9th 2026**.

### Part1: (10%)

**1-** Test the 2 kernel invocations ( **MatmulXrow** and **MatmulYrow** for N=8192 ) implemented in kernel.cu cuda file. (the A, B and C matrices contain **Float** elements).

| Kernel | Total time (Float) |
|---|---|
| MatmulXrow | |
| MatmulYrow | |

**2-** Based on your NVIDIA device characteristics (Architecture, CUDA Cores structure, local memory), do you think that the implementation of the same kernel with integer Matrices (A, B and C) will give the same results? Explain.

**3-** Change the type of matrices A, B and C to **Int** (integer) and test the 2 kernel implementations, for N=8192. Fill the table with value of "*Total execution Time*" (Data Transfer + Kernel execution).

| Kernel | FLOAT | INT(integer) |
|---|---|---|
| MatmulXrow | | |
| MatmulYrow | | |

Compare the performances of the **Float** and **Integer** implementations and interpret the results.

### Part2: (20%)

**1-** Implement the matrix multiplication using the **cuBLAS** Library (Developped by NVIDIA for accelerating HPC application and optimized for NVIDIA architectures).

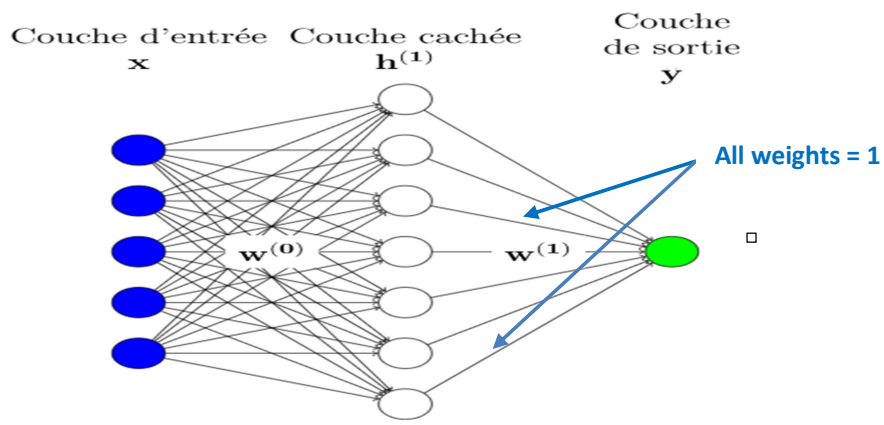
- Give the Host program and the Kernel.
- Compare the performances with the "naïve" implementation (MatmulXrow)

**2-** Explain the principle and the benefits of the **TensorCore** used in NVIDIA Streaming Multiprocessors. Implement the matrix using the tensorCore :

- Explain the implementation (Host program and Kernel).
- Compare the performances with those obtained by cuBLAS Library.

### Part3:

We want to implement a single-hidden-layer neural network on a GPU




The length of the input vector x is 8192. The hidden layer contains 8192 neurons using the sigmoid activation function ($1/1+e^{-x}$). The output layer contains only a single neuron without activation function.

We will split the vector-Matrix multiplication in 2 steps (just for this example. this has not particular interest) : Element by element multiplications executed on CPU and Sums/Activation Functions executed on GPU

**Step 1**: Element by element multiplication (CPU)

$$\begin{pmatrix} x_0 \\ x_1 \\ \cdots \\ \cdot\cdot \\ x_{8191} \end{pmatrix} . \begin{pmatrix} w_{0,0} & w_{0,1} & w_{0,8191} \\ w_{1,0} & w_{1,1} & w_{1,8191} \\ \cdots & \cdot\cdot & \cdot\cdot \\ \cdot\cdot & \cdot\cdot & \cdot\cdot \\ w_{8191,0} & w_{8191,1} & w_{8191,8191} \end{pmatrix} = \begin{pmatrix} x_0 * w_{0,0} & x_0 * w_{0,1} & x_0 * w_{0,8191} \\ x_1 * w_{1,0} & x_1 * w_{1,1} & x_1 * w_{1,8191} \\ \cdots & \cdot\cdot & \cdot\cdot \\ \cdot\cdot & \cdot\cdot & \cdot\cdot \\ x_{8192} * w_{8191,0} & x_1 * w_{8191,1} & x_{8192} * w_{8191,8191} \end{pmatrix}$$

This step will be executed on the **HOST** and the obtained results (8192*8192 float values) will be sent to the GPU:

$$(\boldsymbol{x_0 * w_{0,0}}, \quad \boldsymbol{x_1 * w_{0,1}}, \ldots\ldots, \boldsymbol{x_{8191} * w_{0,8192}}, \quad \ldots\ldots \quad , \quad x_0 * w_{8191,0}, \quad x_1 * w_{8191,1}, \ldots\ldots, x_{8191} * w_{8191,8192})$$

**Step 2**: SUMS/Activation Functions ( on GPU).

$$(\underbrace{x_0 * w_{0,0}, \quad x_1 * w_{0,1}, \ldots\ldots, x_{8191} * w_{0,8192}}_{\Sigma \to \text{Sigmoid} \to \boldsymbol{h_0}}, \underbrace{\ldots\ldots}_{\Sigma \to \text{Sigmoid} \to \boldsymbol{h_i}}, \underbrace{x_0 * w_{8191,0}, \quad x_1 * w_{8191,1}, \ldots\ldots, x_{8191} * w_{8191,8192}}_{\Sigma \to \text{Sigmoid} \to \boldsymbol{h_{8191}}})$$

$$\boldsymbol{y} = \Sigma$$

Obviously, we will be interested in step2 and especially the **sum** operation (SUM REDUCTIOIN).

A straightforward GPU implementation algorithm would be to work in a tree-like fashion: each thread adds two values together, repeatedly halving the number of active threads until only one thread remains with the final sum: **"Naïve" implementation (see table-1 below for n=8)** ***(see Annex-1 for kernel code).***

**Part 3-1 (15%)**

We want to implement the single-hidden-layer neural network using the "naïve" implementation for the sum operation.

**3-1-A** : This implementation is known to be inefficient. What are the key characteristics (disadvantages) that make it so inefficient?

**3-1-B** : To use the naïve implementation Kernel for the single –hidden Layer neural network:

- What code should be added to process the activation function.
- What value will you choose for the Block Size. Explain.
- In the Host program, how will you call the kernel(s).

**3-1-C** : Give the performance of the implementation (execution time on GPU)

**Part 3-2 (55% : *35% for obtained Performance – 20% for report*)**

**3-2-A** Starting from the performances, obtained by the "Naïve ***kernel'***, as a reference value:

- Propose improvements that may increase the Kernel performance (minimum 2 improvements).
- For **each** of the proposed improvements:

  Explain the used technique, how and why it improves the performances. Implement the corresponding kernel in CUDA and give the performance gain obtained

Recommended reading: **https://developer.download.nvidia.com/assets/cuda/files/reduction.pdf**

## Table-1

**Naïve implementation Algorithm with n=8:**

- ***At start****: each thread loads a data.*
- ***At Iteration 1****:only even threads (0,1,2,3) are active,*
- ***At iteration 2*** *only threads, multiples of 4, are active.*
- ***Finally, At Iteration 3*** *only, multiples of 8, are active.*

| thread ID | Start Load | Iter 1 | Iter2 | Iter 3 |
|---|---|---|---|---|
| 0 | elem [**0**] | elem [0] + elem [1] | elem [0] + elem [1] + elem[2] + elem[3] | elem [0] + elem [1] +elem[2] + elem[3] elem[4] + elem[5] + elem[6] + elem[7] |
| 1 | elem [**1**] | inactive | inactive | inactive |
| 2 | elem [**2**] | elem[2] + elem[3] | inactive | inactive |
| 3 | elem [**3**] | inactve | inactive | inactive |
| 4 | elem [**4**] | elem[4] + elem[5] | elem[4] + elem[5] + elem[6] + elem[7] | inactive |
| 5 | elem [**5**] | inactive | inactive | inactive |
| 6 | elem [**6**] | elem[6] + elem[7] | inactive | inactive |
| 7 | elem [**7**] | inactive | inactive | inactive |

# Annex-1

```
__global__ void sumReductionNaive(float* input, float* output, int n) {

    // Shared memory for block-level reduction
    __shared__ float sharedData[256];  // Assuming block size <= 256

    int tid = threadIdx.x;
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    // Load data from global to shared memory
    if (idx < n) {
        sharedData[tid] = input[idx];
    } else {
        sharedData[tid] = 0.0f;
    }

    __syncthreads();  // Ensure all data is loaded

    // Naive reduction within block
    for (int stride = 1; stride < blockDim.x; stride *= 2) {
        if (tid % (2 * stride) == 0) {
            sharedData[tid] += sharedData[tid + stride];
        }
        __syncthreads();  // Synchronize after each step
    }

    // Write block result to output
    if (tid == 0) {
        output[blockIdx.x] = sharedData[0];
    }
}
```